ETSIIT
Escuela Técnica Superior de Ingenierías Informática y de Telecomunicación

Ingeniería Informática + ADE

Universidad de Granada (UGR)

Autor: Ismael Sallami Moreno

Asignatura: Apuntes Visibilidad (PDOO)

Índice

1. Introducción **3**

2. Java **3**
2.1. Private 3
2.2. Paquete 3
2.3. Protected 3

3. Ruby **3**
3.1. Private 4
3.2. Protected 4

4. Ejemplos en Java **4**
4.1. Ejemplo 1 4
4.2. Ejemplo 2 5
4.3. Ejemplo 3 5
4.4. Ejemplo 4 6
4.5. Ejemplo 5 7
4.6. Ejemplo 6 7
4.7. Ejemplo 7 8
4.8. Ejemplo 8 9

5. Ejemplos en Ruby **10**
5.1. Ejemplo 1 10
5.2. Ejemplo 2 10
5.3. Ejemplo 3 11
5.4. Ejemplo 4 11
5.5. Ejemplo 5 12
5.6. Ejemplo 6 12
5.7. Ejemplo 7 13
5.8. Ejemplo 8 14

1 Introducción

- Privado: Solo accesible desde la propia clase.
- Protegido: Solo accesible desde la propia clase y sus subclases.
- Público: Accesible desde cualquier clase.

2 Java

2.1. Private

- Solo accesible desde la propia clase (Instancia y de clase).
- Se puede acceder a elementos privados de otra instancia distinta si es de la misma clase.

2.2. Paquete

- Si no se pone ningún especificador de Visibilidad, por defecto es de paquete.
- son públicos dentro de paquete y privados fuera de él.

2.3. Protected

- Públicos dentro del mismo paquete
- Accesible desde subclases de otros paquetes, es decir, dentro de una instancia se puede acceder a los elementos protegidos de sus superclases, con independencia del paquete en el que se encuentren.
- *Si es de una instancia distinta*:
  - Tienes que ser de la misma clase o una subclase.
  - Debe de ser visible por mi.

3 Ruby

- Atributos siempre **privados**.
- Métodos siempre **públicos** (se pueden cambiar usando modificadores de acceso).
- Initialize es siempre **privado**.

3.1. Private

- Si se hereda se puede acceder a los métodos privados de la superclase (tanto en el ámbito de clase como de instancia).
- No se puede acceder a método privados de clase desde el ámbito de instancia y viceversa.

3.2. Protected

- Se puede acceder a los métodos protegidos de la superclase desde la subclase.
- No existen métodos protegidos de clase.

En ruby **una clase y su instancia no son de la misma clase**.

Los atributos @@ **si pueden ser accedidos desde ámbito de instancia**.

4 Ejemplos en Java

Se especifican las soluciones en forma de comentarios.

4.1. Ejemplo 1

```java
package unPaquete;

public class Padre {
    private int privado;
    protected int protegido;
    int paquete;
    public int publico;

    public void testInstanciaPadre(Padre o) {
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido);
        System.out.println(o.paquete);
        System.out.println(o.publico);
    }

    public static void testClasePadre(Padre o) {
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido);
        System.out.println(o.paquete);
        System.out.println(o.publico);
    }
}
```

4.2. Ejemplo 2

```java


package unPaquete;

public class HijaPaquete extends Padre {

    public void testInstanciaHijaPaquete(Padre o) {
        // No se puede acceder a 'privado' directamente ni a través de una instancia
        // System.out.println(privado);
        // System.out.println(o.privado);

        System.out.println(protegido); // Se puede acceder porque es protegido
        System.out.println(o.protegido); // Se puede acceder porque es protegido

        System.out.println(o.paquete); // Se puede acceder porque está en el mismo paquete
        System.out.println(o.publico); // Se puede acceder porque es público
    }

    public static void testClaseHijaPaquete(Padre o) {
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);

        System.out.println(o.protegido); // Se puede acceder porque es protegido
        System.out.println(o.paquete); // Se puede acceder porque está en el mismo paquete
        System.out.println(o.publico); // Se puede acceder porque es público
    }
}
```

4.3. Ejemplo 3

```java


package otroPaquete;

public class HijaOtroPaquete extends Padre {

    public void testInstanciaHijaOtroPaquete(Padre o) {
        // Acceso a elementos heredados
        // No se puede acceder a 'privado' directamente
        // System.out.println(privado);
        // No se puede acceder a 'paquete' porque no está en el mismo paquete
        // System.out.println(paquete);
```

```
        System.out.println(protegido); // Se puede acceder porque es protegido

        // Acceso a elementos de otra instancia
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido); // Se puede acceder porque es protegido
        // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
        // System.out.println(o.paquete);
        System.out.println(o.publico); // Se puede acceder porque es público
    }

    public static void testClaseHijaOtroPaquete(Padre o) {
        // Acceso a elementos de otra instancia
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido); // Se puede acceder porque es protegido
        // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
        // System.out.println(o.paquete);
        System.out.println(o.publico); // Se puede acceder porque es público
    }
}
```

4.4. Ejemplo 4

```

package otroPaquete;

public class HijaOtroPaquete extends Padre {

    // El mismo código cambiando solo el tipo del parámetro
    public void testInstanciaHijaOtroPaquete(HijaOtroPaquete o) {
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido); // Se puede acceder porque es protegido
        // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
        // System.out.println(o.paquete);
        System.out.println(o.publico); // Se puede acceder porque es público
    }

    public static void testClaseHijaOtroPaquete(HijaOtroPaquete o) {
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido); // Se puede acceder porque es protegido
```

```
        // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
        // System.out.println(o.paquete);
        System.out.println(o.publico); // Se puede acceder porque es público
    }
}
```

4.5. Ejemplo 5

```

package otroPaquete;

public class HijaOtroPaquete extends Padre {

    // El mismo código. El tipo del parámetro es subclase.
    public void testInstanciaHijaOtroPaquete(NietaOtroPaquete o) {
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido); // Se puede acceder porque es protegido
        // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
        // System.out.println(o.paquete);
        System.out.println(o.publico); // Se puede acceder porque es público
    }

    public static void testClaseHijaOtroPaquete(NietaOtroPaquete o) {
        // No se puede acceder a 'privado' de otra instancia
        // System.out.println(o.privado);
        System.out.println(o.protegido); // Se puede acceder porque es protegido
        // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
        // System.out.println(o.paquete);
        System.out.println(o.publico); // Se puede acceder porque es público
    }
}

// NietaOtroPaquete deriva de HijaOtroPaquete (ambas están en otroPaquete)
```

4.6. Ejemplo 6

```
    package otroPaquete;

    public class NietaOtroPaquete extends HijaOtroPaquete {

        // Ahora probamos con un parámetro de la superclase
        public void testInstanciaNietaOtroPaquete(HijaOtroPaquete o) {
            // No se puede acceder a 'privado' de otra instancia
```

```
            // System.out.println(o.privado);
            // No se puede acceder a 'protegido' de otra instancia en un paquete diferente
            // System.out.println(o.protegido);
            // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
            // System.out.println(o.paquete);
            System.out.println(o.publico); // Se puede acceder porque es público
        }

        public static void testClaseNietaOtroPaquete(HijaOtroPaquete o) {
            // No se puede acceder a 'privado' de otra instancia
            // System.out.println(o.privado);
            // No se puede acceder a 'protegido' de otra instancia en un paquete diferente
            // System.out.println(o.protegido);
            // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
            // System.out.println(o.paquete);
            System.out.println(o.publico); // Se puede acceder porque es público
        }
    }
```

4.7. Ejemplo 7

```
    package otroPaquete;

    public class NietaOtroPaquete extends HijaOtroPaquete {

        public void testInstanciaNietaOtroPaquete(NietaOtroPaquete o) {
            // No se puede acceder a 'privado' de otra instancia
            // System.out.println(o.privado);
            System.out.println(o.protegido); // Se puede acceder porque es protegido
            // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
            // System.out.println(o.paquete);
            System.out.println(o.publico); // Se puede acceder porque es público
        }

        public static void testClaseNietaOtroPaquete(NietaOtroPaquete o) {
            // No se puede acceder a 'privado' de otra instancia
            // System.out.println(o.privado);
            System.out.println(o.protegido); // Se puede acceder porque es protegido
            // No se puede acceder a 'paquete' de otra instancia porque no está en el mismo paquete
            // System.out.println(o.paquete);
            System.out.println(o.publico); // Se puede acceder porque es público
        }
```

```
    }
```

4.8. Ejemplo 8

```
    // En el paquete base
    package base;

    public class A {
        protected int protegidoA = 0;
    }

    // En el paquete base
    public class B extends A {
        protected int protegidoB = 1;
    }

    // En el paquete base2
    package base2;
    import base.*;

    public class C extends B {
        protected int protegidoC = 2;

        public void test() {
            A a = new A();
            // No se puede acceder a 'protegidoA' de una instancia de A en un paquete diferente
            // a.protegidoA = 666;

            B b = new B();
            // No se puede acceder a 'protegidoB' de una instancia de B en un paquete diferente
            // b.protegidoB = 666;

            C c = new C();
            c.protegidoA = 555; // Se puede acceder porque C hereda de A
            c.protegidoB = 555; // Se puede acceder porque C hereda de B

            D d = new D();
            d.protegidoA = 555; // Se puede acceder porque D hereda de A
            d.protegidoB = 555; // Se puede acceder porque D hereda de B
            d.protegidoD = 555; // Se puede acceder porque es miembro de D

            D d2 = new D();
            d2.protegidoB = 555;
            // No se puede acceder a 'protegidoD' de otra instancia en un paquete diferente
            // d2.protegidoD = 555;

            this.protegidoA = 777; // Se puede acceder porque C hereda de A
            this.protegidoB = 777; // Se puede acceder porque C hereda de B
            this.protegidoC = 777; // Se puede acceder porque es miembro de C
        }
```

```
    }

    // En el paquete base2
    package base2;

    public class D extends C {
        protected int protegidoD = 3;
    }
```

5 Ejemplos en Ruby

5.1. Ejemplo 1

```
    class Padre
      private
      def privado
      end

      protected
      def protegido
      end

      public
      def publico
      end

      def test(p)
        privado
        self.privado # Correcto solo a partir de Ruby 2.7
        # p.privado # No se puede acceder a un método privado de otra
            instancia

        protegido
        self.protegido
        p.protegido # Se puede acceder a un método protegido de otra
            instancia
      end
    end
```

5.2. Ejemplo 2

```
    # Fuera de cualquier clase

    Padre.new.test(Padre.new)
    p = Padre.new

    # Acceso a métodos fuera de la clase o subclases

    # p.privado # No se puede acceder a un método privado fuera de la clase
    # p.protegido # No se puede acceder a un método protegido fuera de la
        clase o subclases
```

```
p.publico # Se puede acceder a un método público
```

5.3. Ejemplo 3

```
class Hija < Padre
  def test(p)
    privado
    self.privado # Correcto solo a partir de Ruby 2.7
    # p.privado # No se puede acceder a un método privado de otra instancia

    protegido
    self.protegido
    p.protegido # Se puede acceder a un método protegido de otra instancia

    publico
    self.publico
    p.publico
  end
end

# Fuera de cualquier clase

Hija.new.test(Hija.new)
Hija.new.test(Padre.new)
h = Hija.new
# h.privado # No se puede acceder a un método privado fuera de la clase
# h.protegido # No se puede acceder a un método protegido fuera de la clase o subclases
h.publico # Se puede acceder a un método público
```

5.4. Ejemplo 4

```
class Padre
  private
  def privado_instancia
  end

  def self.privado_clase
  end

  private_class_method :privado_clase

  public
  def test
    # No se puede acceder a un método privado de clase con un receptor explícito
    # self.class.privado_clase
  end

  def self.test(p)
```

```
      # No se puede acceder a un método privado de instancia de otra instancia
      # p.privado_instancia
    end
  end

  # Fuera de cualquier clase

  Padre.new.test
  Padre.test(Padre.new)
```

5.5. Ejemplo 5

```
  class Padre
    @instanciaDeClase = 1
    @duda = 2
    @@deClase = 11
    @@duda = 22

    def initialize
      @deInstancia = 333
      @duda = 444
    end

    def self.salida
      # puts @instanciaDeClase + 1
      # No se puede acceder a la variable de instancia de la clase desde una subclase
      # puts @duda + 1 unless @duda.nil? # desde Hija?
      puts @@deClase + 1
      puts @@duda + 1
    end

    def salida
      #puts @deInstancia + 1
      #puts @duda + 1
      # No se puede acceder porque en Ruby todos los atributos son privados
      puts @@deClase + 1
      puts @@duda + 1
    end
  end
```

5.6. Ejemplo 6

```
  class Hija < Padre
    @instanciaDeClase = -1

    # Sobreescribimos el valor fijado anteriormente
    # Este atributo es compartido
    @@deClase = -11

```

```
      def modifica
        # Acceso a los atributos definidos en Padre
        # No se puede acceder a la variable de instancia de la clase desde
            una instancia
        # @duda += 111
      end #deberia de ser self.modifica
    end

    # Fuera de cualquier clase

    Padre.salida
    Hija.salida # Atención a lo que ocurre con la segunda línea

    p = Padre.new
    p.salida
    h = Hija.new
    h.salida

    # h.modifica debería de ser self.modifica
    h.salida
```

5.7. Ejemplo 7

```
    class A
      protected
      def protegidoA
      end
    end

    class B < A
      protected
      def protegidoB
      end
    end

    class C < B
      protected
      def protegidoC
      end

      public
      def test
        A.new.protegidoA
        B.new.protegidoA
        B.new.protegidoB
        C.new.protegidoA
        C.new.protegidoB
        C.new.protegidoC
        D.new.protegidoA
        # D.new.protegidoD
        # No puedo debido a que estoy en C y es D la que hereda de C, puedo
            acceder de D a C, pero no de C a D
      end
    end
```

```

class D < C
  protected
  def protegidoD
  end
end

C.new.test
# C.new.protegidoC
# No se puede acceder a un método protegido fuera de la clase o
    subclases
```

5.8. Ejemplo 8

```
class FalsaSeguridad
  # Un consultor puede ser muy peligroso
  attr_reader :lista

  def initialize
    @lista = [1, 2, 3, 4]
  end

  def info
    puts @lista.size
  end
end

# Fuera de cualquier clase
f = FalsaSeguridad.new
f.info # 4

# Modificamos el estado interno
# Simplemente usando un consultor
# Aunque el atributo sea privado
# Cuidado con las referencias
f.lista.clear

f.info # 0
```